PROJECT REVIEW

SSH Monitor UI

Idle Testing Monitor — a real-time web application to monitor multiple Linux/Windows systems over SSH, collecting CPU idle-state and power data on a timed schedule.

Node.js + Express | WebSocket | SSH2 | Vanilla JS Frontend | PMUtil + Turbostat | File-based Logging

TABLE OF CONTENTS

1 Project Overview
2 At a Glance
3 Key Features
4 Tech Stack
5 Architecture
6 How It Works
7 File Structure
8 Frontend Pages
9 API Reference
10 Configuration
11 SSH Resilience
12 Quick Start

SECTION 1

Project Overview

SSH Monitor UI (branded as *Idle Testing Monitor*) is a full-stack web application designed for Intel's hardware validation teams. Its purpose is to remotely connect to multiple servers via SSH, run **PMUtil** and **Turbostat** diagnostic commands on a repeating schedule, and surface the collected CPU C-state / power data in a live dashboard — all without needing to manually log into each machine.

> **Core use-case:** An engineer wants to verify that several test systems enter deep CPU idle states (C6, PC6) during an "idle" soak test. They add each system's IP and credentials to the UI, set an idle interval and total duration, hit *Add & Monitor*, and the tool handles the rest — collecting, parsing, logging, and displaying data in real time.

The app is built entirely on open-source Node.js packages and runs directly on Windows (with `start.bat`) or any platform that has Node.js. No database is required — all persistent data is stored in log files under the `logs/` directory.

SECTION 2

At a Glance

~25KB Backend (server.js)	**5** Frontend files	**8** REST API endpoints
WS Real-time updates	**4** npm dependencies	**∞** Systems (concurrent)

SECTION 3

✨ Key Features

Multi-system Monitoring

Add unlimited systems. Each system runs its own independent monitoring loop concurrently.

Configurable Scheduling

Per-system idle interval and total monitor duration. Target run count is auto-calculated.

Real-time WebSocket UI

Live dashboard updates via WebSocket — no page refresh needed. Data streams as it arrives.

SSH with Auto-Recovery

If SSH drops, the backend automatically retries. It pauses the timer and resumes when the connection is restored — for up to 1 hour.

C6 / PC6 Parsing

Automatically extracts CPU%c6 and Pkg%pc6 values from Turbostat output tables, plus socket count from PMUtil.

Timestamped Log Files

Every loop's PMUtil and Turbostat output is appended to a timestamped log file per system.

Role-based Access

User vs Admin roles. Admins can stop monitors and clear history. Admin login is gated by password.

Live Output Window

Each system has a dedicated live output page (system.html) that shows raw PMUtil and Turbostat text in real time.

Portable via Env Vars

PORT, HOST, LOGS_DIR, SSH credentials, and default commands — all configurable via environment variables.

SECTION 4

Tech Stack

Node.js
JavaScript runtime powering the backend server

Express 4
HTTP server, REST API routing, and static file serving

ssh2
Pure-JS SSH client. Opens connections and executes remote commands

ws (WebSocket)
Server-side WebSocket for pushing live updates to all browser clients

Vanilla HTML/CSS/JS
No framework. Pure browser-side code for maximum portability

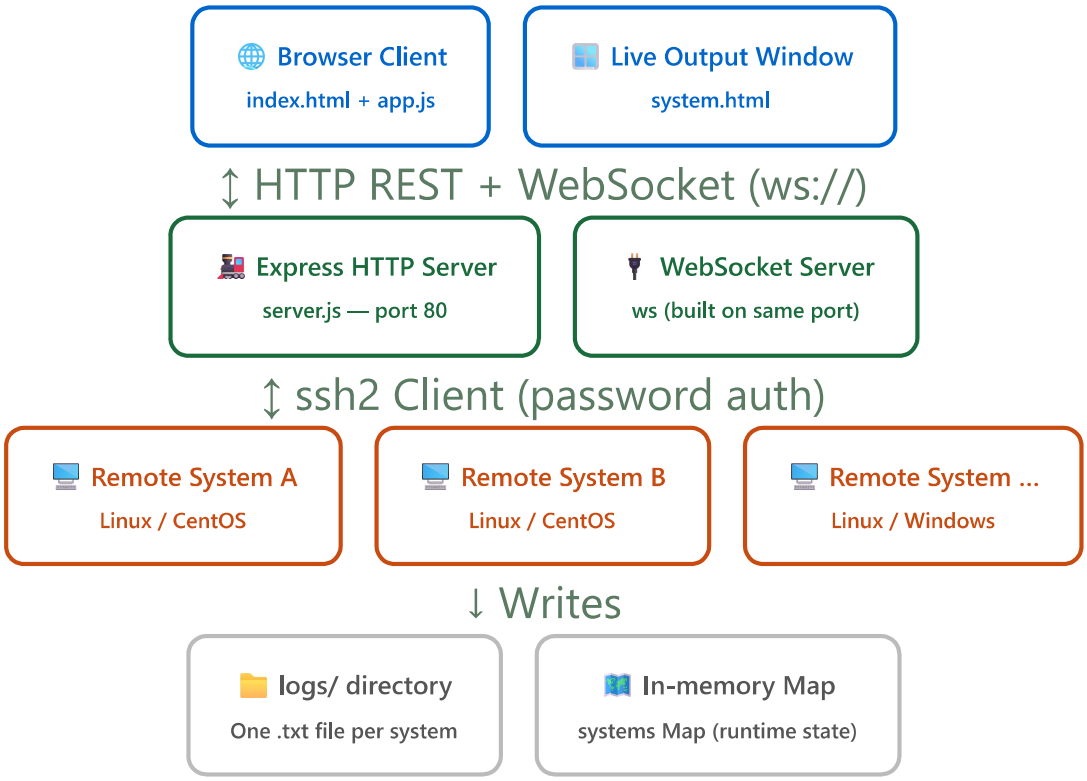
Node.js fs/promises
Async file I/O for appending log blocks per system loop

cors
Cross-Origin Resource Sharing support for flexible deployment

nodemon (dev)
Hot-reload during development — auto-restarts on file changes

SECTION 5

Architecture Diagram



> **Key design choice:** The server holds all system state in a JavaScript `Map` in memory. There is no database. Logs are the only persistent storage. This makes the app extremely simple to deploy but means state is lost on server restart.

SECTION 6

How It Works — Data Flow

1 User Adds a System

User fills in system name, IP, SSH credentials, idle time, and monitor duration on the main page and clicks *Add & Monitor*. The browser POSTs to `POST /api/systems` which registers the system in the in-memory `Map` and immediately starts monitoring.

2 Backend Starts the Monitoring Loop

`startMonitoring()` initialises all counters, marks status as `running`, and calls `scheduleNextRun()` to schedule the first data collection using `setTimeout`. The loop count and target runs are computed from the idle / duration settings.

3 SSH Connection & Command Execution

On each loop tick, `collectOnce()` opens a fresh SSH connection via `ssh2`. It executes the **PMUtil** command first, then the **Turbostat** command sequentially, capturing stdout from each. Each command has a 15-second timeout.

4 Parsing & Extraction

`parseSockets()` extracts the socket count from PMUtil output via regex. `parseTurboStat()` locates the header row containing `CPU%c6` and `Pkg%pc6` in the tabular output, then reads values from the first data row — this is the key metric for idle state validation.

5 Logging to File

Raw PMUtil and Turbostat output is appended to a per-system timestamped log file under `logs/`. Each block is prefixed with an ISO timestamp and a label like `LOOP_3_PMUTIL`, making logs human-readable and machine-parseable.

6 WebSocket Broadcast

After each collection, the backend calls `broadcast()` which sends two events to all connected browsers: a `systemUpdate` (summary stats) and a `systemOutput` (raw text). Both the main dashboard and any open live-output windows receive these in real time.

7

Loop Continues or Stops

After each successful collection, the loop counter increments and `scheduleNextRun()` is called again using the planned tick time (aligned to the original start time) to prevent drift. When `loopCount >= targetRuns`, the system is marked `completed`.

SECTION 7

File Structure

📦 **project root/**

- ├── `server.js` ⭐ Main backend — Express + WebSocket + SSH monitoring logic (~25 KB)
- ├── `package.json` npm metadata, scripts (start / dev), and 4 dependencies
- ├── `.env.example` Template for environment variable configuration
- ├── `start.bat` Windows one-click launcher
- ├── `README.md` Setup instructions and API reference
- ├── 📁 **public/** All static frontend assets (served by Express)
  - │ ├── `index.html` ⭐ Main dashboard — Add System form, Active Summary, History
  - │ ├── `app.js` ⭐ Frontend logic — role management, WebSocket, API calls, table rendering
  - │ ├── `styles.css` Shared CSS for all pages — green theme, cards, tables, modals
  - │ ├── `system.html` Per-system live output page — shows PMUtil/Turbostat raw text + loop summary
  - │ └── `one-page-summary.html` Printable/shareable summary view
- └── 📁 **logs/** Auto-created at runtime — one .txt log file per monitored system

> **Naming convention for log files:** `<systemName>_<ip>_<timestamp>.txt`
> Each file contains timestamped blocks for every loop: `LOOP_N_PMUTIL`, `LOOP_N_TURBOSTAT`, and any `ERROR` or `SSH_RECOVERY` entries.

SECTION 8

Frontend Pages

index.html — Main Dashboard

The primary view. Contains:

- **Add System form** (name, IP, OS type, credentials, idle time, duration)
- **Active Summary table** (live updating, shows Sockets, C6, PC6, elapsed, time left, SSH retries)
- **History table** (all stopped/completed systems with start/end times and stop reason)
- **Role selector** (User / Admin)
- **Log modal** (view or download per-system log, with loop-wise C6/PC6 table)

system.html — Live Output

Opens in a new tab for any selected system. Shows:

- Raw PMUtil output (latest)
- Raw Turbostat output (latest)
- Loop-wise C6/PC6 summary table (parsed from log file)
- Link to download the full log

Updates every 5 seconds and via WebSocket.

one-page-summary.html

A compact, printable single-page overview. Designed for reporting or sharing test results.

app.js Responsibilities

- ✓ Connects to the backend WebSocket and handles `systemUpdate` / `systemOutput` / `tick` messages
- ✓ Renders and updates the Active Summary and History tables dynamically
- ✓ Manages role state (User/Admin) with localStorage persistence and Admin login modal
- ✓ Handles the Add System form submission (`POST /api/systems`)
- ✓ Opens the log modal, fetches log text, and parses loop-wise C6/PC6 summary from it

- ✓ Stop button sends `POST /api/systems/:id/stop` (Admin only)
- ✓ Clear History button (Admin only) removes completed/stopped entries from the summary

SECTION 9

API Reference

METHOD	ENDPOINT	DESCRIPTION	RESPONSE
GET	/health	Health check — confirms server is running	{ status: "ok" }
GET	/api/systems	List all systems (including stopped/completed) with their full summary view	Array of system summary objects
GET	/api/systems/active	List only currently running systems	Array of running system summaries
POST	/api/systems	Register a new system and start monitoring. Body: { systemName, ip, username, password, idleTimeSec, monitorDurationSec, … }	Created system summary
POST	/api/systems/:id/start	Re-start monitoring on an existing (stopped/completed) system	Updated system summary
POST	/api/systems/:id/stop	Manually stop a running system (Admin action)	Updated system summary with status: "stopped"

METHOD	ENDPOINT	DESCRIPTION	RESPONSE
GET	/api/systems/:id/details	Get the latest raw PMUtil and Turbostat output for a system (used by system.html)	System summary + pmutilOutput + turbostatOutput
GET	/api/systems/:id/log	Download/view the full text log file for a system	Plain text log file

WebSocket Events

EVENT TYPE	DIRECTION	PAYLOAD
systemUpdate	Server → Client	Full system summary object — triggers table row refresh on dashboard
systemOutput	Server → Client	Raw PMUtil + Turbostat text for a specific system ID — used by system.html
tick	Server → Client	Periodic heartbeat with elapsed/remaining time updates for all active systems

SECTION 10

Configuration (Environment Variables)

VARIABLE	DEFAULT	DESCRIPTION
`PORT`	`80`	HTTP port the server listens on
`HOST`	`0.0.0.0`	Bind address — use machine IP to share over LAN
`LOGS_DIR`	`./logs`	Directory where log files are stored (absolute or relative)
`SSH_USERNAME`	*empty*	Global fallback SSH username when per-system field is blank
`SSH_PASSWORD`	*empty*	Global fallback SSH password when per-system field is blank
`DEFAULT_PMUTIL_COMMAND`	`cd /root/apps/PI_PM/pmutil && ./pmutil_bin -i`	Override the default PMUtil shell command
`DEFAULT_TURBOSTAT_COMMAND`	`./turbostat -q -s … -interval 1 -n 5`	Override the default Turbostat shell command

> ⚠️ **Security note:** SSH passwords are transmitted in plain text from the browser to the backend. Ensure the app runs on a trusted internal network. Do not expose port 80 to the public internet. The `SSH_PASSWORD` env var keeps credentials out of the browser form, but it is shared across all systems.

SECTION 11

SSH Resilience & Recovery Logic

One of the most important backend features is the automatic SSH recovery system:

! SSH Failure Detected

If a connection attempt or heartbeat fails, `sshFailureSince` is set to the current timestamp. The timer is paused (elapsed time tracking excludes the pause duration).

↺ Retry Every 1 Second

`probeSshRecovery()` is called every **1 second** using a short `SSH_RECOVERY_RETRY_MS` timeout. This aggressively probes for reconnection.

✓ Recovery Detected

On success, `markSshRecovered()` calculates the total pause duration, increments `sshReattemptCount` (if paused > 10 s), resets all failure flags, and logs a recovery message. The UI shows a green recovery banner for 8 seconds.

✗ 1-Hour Timeout Limit

If SSH remains unreachable for more than **1 hour** (`SSH_MAX_RECOVERY_DURATION_MS`), the system is permanently stopped with status `error` and the reason "SSH unreachable for over 1 hour".

> A separate **SSH heartbeat** also runs every **5 seconds** during normal operation to proactively detect SSH drops even between scheduled loops. This ensures failures are reported immediately without waiting for the next data collection.

SECTION 12

🚀 Quick Start

1 **Install dependencies**

`npm install` — installs express, ssh2, ws, cors

2 **Configure (optional)**

Copy `.env.example` to `.env` and set `SSH_USERNAME`, `SSH_PASSWORD`, etc. Or set env vars inline.

3 **Start the server**

`npm start` (production) or `npm run dev` (nodemon watch mode). On Windows you can also double-click `start.bat`.

4 **Open the UI**

Navigate to `http://localhost` (or `http://<your-ip>:PORT` from another machine on the same network).

5 **Add a system and monitor**

Fill in the system name, IP, SSH credentials, set idle time and monitor duration, then click **Add & Monitor**. Watch data populate in the Active Summary table in real time.